CogniFlow - Demo Video Guide

Everything you need to record the 3-5 minute demo video for the Agentic AI Hackathon, Tech Zephyr 4.0. Written for Team BloodCoded.

You are the only speaker

This video has one narrator, and that is you. You will be doing two jobs at once - driving the terminal and explaining what appears. That is harder than it sounds, so this guide is built around making it manageable: learn one idea properly, know where each line comes from, and read the script for the rest.

There are two documents. This one teaches you the concepts and tells you *when* to say things. The second - **CogniFlow_Video_Script.pdf** - contains the actual words, split into numbered PARTs. Keep both open.

How the two documents fit together

This guide (PDF 1)	The script (PDF 2)
Explains what everything means	Gives you the exact words
Tells you which PART to read and when	Is organised into PART 0 to PART 10
Read it over the next few days	Read it while recording
Sections 1-4 teach; 5-9 prepare you	Blue panels are the only things spoken

What is in this guide

Section	What it is	When to read it
0. Getting the code	Putting CogniFlow on YOUR machine	Before anything else
1. The one idea	What CogniFlow does, in plain English	First. Twice.
2. The five concepts	Every term you might be asked about	Over 2-3 sittings
3. Reading the screen	What each line of output means	With the app open
4. The map	Which PART to read at which moment	Before rehearsing
5. Narrating alone	Technique for doing both jobs at once	Before rehearsing
5b. Recording it	Software, settings, joining clips	Before recording
6. Setup	Exact commands, in order	Recording day
7. If it breaks	What to do live	Skim, then trust it
8. Questions	What judges may ask you	Before the finale
9. Glossary	Every term in one place	Whenever stuck
10. Learning plan	What to study, in priority order	Today

0. Getting CogniFlow running on your machine

You are a collaborator on the GitHub repository, but the code is not on your computer yet, and everything else in this guide assumes it is. This section assumes you have never done this before and that nobody is sitting next to you. Follow it in order. Every step says what you should see, and what to do when you see something else instead.

Budget an evening - and not recording day

The commands themselves take about five minutes. What takes longer is the **79 MB** download in step 0.7 and any installing you need to do first. Do all of section 0 **the day before you record.** Once done it stays done, and every later run starts in seconds.

0.1 What you need before you start

You need	Why	If you do not have it
Git	To download the code	git-scm.com - install with all defaults
Python 3.11 or newer	To run it	python.org/downloads - during setup TICK 'Add python.exe to PATH'. This matters; without it nothing below works.
Internet	Downloads, and the AI providers	-
About 1 GB free disk	Libraries and the search model	-

0.2 Opening a terminal

A terminal is a window where you type commands. On Windows press **Windows key + R**, type `powershell`, press Enter. That window is your terminal.

- **You can paste into it.** Copy a command from this PDF, then right-click in the terminal. Ctrl+V also works in PowerShell.
- **It runs commands where it is standing.** After 0.4 you move into the project folder, and every later command must be typed there. If you close the terminal, you must cd back into the folder before continuing.

0.3 Check what you already have

Type these, pressing Enter after each:

```
git --version
python --version
```

What you see	What it means
A version number for both	Good. Go to 0.4.
'not recognized' for git	Install Git from git-scm.com, then CLOSE the terminal and open a new one.
'not recognized' for python	Install Python from python.org and tick 'Add python.exe to PATH'. Then close and reopen the terminal.

Python 3.10 or older	Install a newer one from python.org. Built on 3.14; 3.11 and up are fine.
It opens the Microsoft Store	Windows is intercepting the command. Install from python.org instead, then reopen the terminal.

Closing and reopening the terminal after installing anything is not optional. A terminal only learns about newly installed programs when it starts.

0.4 Download the code

```
cd Desktop
git clone https://github.com/sohan1611/CogniFlow.git
cd CogniFlow
```

You should see it downloading, then finish. You are now standing inside the project folder, and every remaining command is typed here.

If it says *Repository not found*: accept the collaborator invitation in your GitHub notifications first, then try again.

0.5 Install the libraries

```
python run.py install
```

This creates a private folder of libraries *inside the project*. It changes nothing else on your computer and takes about a minute. Yellow WARNING lines about scripts not being on PATH are normal - ignore them.

What you see	What to do
It finishes with no red text	Good. Continue to 0.6.
'No module named pip'	Reinstall Python from python.org, ticking pip in the optional features.
A red error naming a specific package	Run the command again - a dropped download is the usual cause. If it fails identically twice, see 0.9.

0.6 Get your own API keys

Make your own keys - do not ask for anyone else's

The keys are deliberately not in the repository. Committing credentials is forbidden by rulebook section 6 and is a disqualification path under section 13, so cloning gives you the code and no keys. Both providers are free, take about two minutes each, and ask for no billing details.

Your own keys are also better for the team: two accounts have separate rate limits, so you get double the headroom instead of sharing one ceiling.

Provider	Where to get a free key	Name in .env
Groq	console.groq.com - sign in, API Keys, Create API Key. Copy it immediately; it is shown once.	GROQ_API_KEY
Google	aistudio.google.com/apikey - sign in, Create API key.	GOOGLE_API_KEY

Putting the keys in the right place

The project reads them from a file called `.env` - note the dot at the start. A template sits next to it called `.env.example`. Make the real one from your terminal, inside the project folder:

```
copy .env.example .env
notepad .env
```

Notepad opens. Find these two lines and paste your keys directly after the equals signs - **no quotes, no spaces**:

```
GROQ_API_KEY=gsk_yourkeyhere
GOOGLE_API_KEY=AIzaYourKeyHere
```

Save with Ctrl+S and close Notepad.

> **Do not create this file by hand in File Explorer**
>
> Windows hides file extensions by default, so a file you name '.env' in Explorer usually becomes '.env.txt' and the project will never find it. The `copy` command above avoids that entirely.

Check the keys work, on their own

```
python run.py check
```

This makes one real call per provider. You want two lines starting `[PASS]`. A line saying `[SKIP] anthropic` is expected and correct - we do not use a paid provider.

What you see	What it means
[PASS] groq and [PASS] google	Both keys work. Continue.
[SKIP] for a key you did set	The name in .env is wrong, or the file did not save. Check the spelling is exactly GROQ_API_KEY and GOOGLE_API_KEY.
An authentication or 401 error	The key was copied incompletely. Generate a fresh one and paste it again.
[PASS] for only one of the two	You can still record, but expect occasional degraded=True. The second key is two minutes and worth it.

0.7 Build the search index

```
python run.py ingest
```

> **The slow step - the one to do the day before**
>
> The first run downloads a **79 MB** model used to search the course notes. Free, and it happens exactly once. On a new machine it can take several minutes and may look frozen while downloading - it is not. Every later run reuses it and finishes in seconds.

It finishes by listing each topic with a chunk count, like `recursion: 8`. That is success.

0.8 Prove the whole thing works

```
python run.py live
```

This is the real demo - the one you will record. It takes roughly 40 to 70 seconds. At the end you want **both** of these:

Look for	Exactly

The learning path	Learning path : recursion -> functions -> recursion
The self-checks	seven lines starting [PASS], and no [FAIL]

If you see both, **you are completely set up** and the rest of this guide applies exactly as written. Run it twice more to get familiar with what scrolls past, and read section 6 - it explains what may legitimately differ between runs and what must not.

0.9 If something is still broken

Work down this list. Almost every problem is one of the first three.

Symptom	Most likely cause	What to do
A command is 'not recognized'	Not installed, or the terminal was open before you installed it	Close the terminal, open a new one, retry. If it still fails, install the program per 0.1.
'cannot find the path' / 'No such file'	You are not standing in the project folder	Type 'cd Desktop' then 'cd CogniFlow', and retry.
degraded=True in the output	A key is missing, misspelled, or wrapped in quotes	Run 'python run.py check' and fix whatever it reports, per 0.6.
ModuleNotFoundError	The install did not finish	Run 'python run.py install' again, watching for red text.
Rate limit, or 429, in the output	You ran the demo several times in quick succession	Wait a minute and run again. With both keys it usually recovers on its own - that is the failover working, and it is worth mentioning on camera.
It runs but no [PASS] lines appear	The run stopped early	Scroll up and read the last lines before it stopped. If that does not help, use 0.10.
Works, but very slow the first time	The 79 MB download in 0.7	Normal, and only once. Let it finish.

0.10 If you cannot get a full live run today

You still have a video. These are ranked best to worst, and **all four are honest** - none of them fabricates anything.

Tier	What it is	What to say on camera
1	Both keys, full live run. What the script assumes.	Nothing extra.
2	One key only. Works; occasional degraded=True.	"That provider is rate-limited there, so it fell back to a deterministic template. The tutoring decisions are unaffected - the model does not make those."
3	No keys at all. Run 'python run.py verify' instead. Everything works except the exercise wording, which comes from built-in templates.	"I am running this without an API key, so the exercise wording comes from a template. Every decision you are about to see - the diagnosis, the redirect, the mastery updates - is computed exactly as it would be live." Then narrate normally.
4	Setup will not cooperate at all. Ask Sohan for five minutes: he runs 'python run.py live', screen-records it, and sends you the file.	Record your narration over that footage. Section 5 explains why narrating over a finished real run is legitimate.

Tier 3 beats no video, by a mile

A required deliverable that is missing scores zero. A working demo narrated honestly, with one sentence explaining that the exercise wording is templated, loses very little - the agentic behaviour a judge is assessing is identical either way. **Do not let a missing API key stop you recording.**

1. The one idea

If you remember nothing else, remember this page. It is the entire project, and it is what PART 1 of the script says in your own voice.

The situation

A student is learning Python. They try a recursion exercise and get it wrong. They try again and get it wrong again.

What every other AI tutor does

It gives them an **easier recursion question**. Maybe it adds a hint. Maybe it explains recursion again, more slowly.

Why that fails

Very often the student's real problem is not recursion at all. It is that they do not understand what `return` does when one function calls another. That is a **functions** problem. Recursion is just where it became visible. So the easier recursion question fails too, for exactly the same reason, and the student concludes they are 'bad at recursion' when nobody ever taught them the thing recursion is built on.

What CogniFlow does instead

It notices the pattern, works out that the real gap is **functions**, **changes its own goal** to teach functions, teaches that, and then comes back to recursion - which it never forgot was the original target.

```
recursion  ->  functions  ->  recursion
(failing)      (the real      (back to what
                gap)           they wanted)
```

The sentence to memorise

"Most AI tutors personalise **which question** you get. CogniFlow personalises **the route through the knowledge**."

Everything else here is detail underneath that sentence. If a judge only hears one thing from you, this is the one.

2. The five concepts

These are the five things a judge might ask about. Each has: what it is, why it matters, and the one sentence to say. You do not need any mathematics.

2.1 The prerequisite graph (the core)

What it is. A map of which topic depends on which. You cannot understand recursion without functions. You cannot understand functions without variables. We wrote that map down as a structure the program can walk through.

```
variables  ->  functions  ->  recursion  ->  recursion_tree
variables  ->  loops      ->  nested_loops
```

Why it matters. It lets the program ask a question no chatbot can ask: 'this student keeps failing recursion - what does recursion *depend on*, and are they weak at that instead?'

Where it appears: PART 4, on the `[prereq_redirect]` line.

Say:

> *"It walks a prerequisite graph. Recursion depends on functions, so when recursion keeps failing it checks whether functions is the real problem."*

2.2 Mastery, and why it is not a score out of ten

What it is. For every skill we keep a number between 0 and 1 - our best estimate of the chance the student actually knows it. Recursion at 0.35 means "we think there is about a 35% chance they have got it".

The technique is **Bayesian Knowledge Tracing**, from a 1995 paper by Corbett and Anderson. You do not need the equations. You need to know why it beats counting right answers:

- It knows people **guess**. One lucky correct answer does not prove mastery.
- It knows people **slip**. One careless mistake does not erase what they know.
- It tracks **confidence** separately. Being 80% sure after 3 questions is not the same as after 30, and the program treats those differently.

Where it appears: PART 3 and PART 6, on the `[mastery]` lines.

Say:

> *"Mastery is a Bayesian estimate, not a running total. It models guessing and slipping, and it produces a confidence number the routing logic actually uses."*

2.3 "The model proposes, the guard disposes"

What it is. Our architecture in five words, and the thing the team is proudest of. An AI model suggests what to do next - retry, move on, go back a step. It does not get the final say. Separate, ordinary, predictable code called **the guard** checks that suggestion against fixed rules and overrules it when it breaks one.

A rule the guard enforces: *you may not send a student onward if their mastery is below 0.6.* The AI might suggest it. The guard says no.

Why it matters - three reasons, worth learning properly:

Reason	What it buys us

Safety	The AI cannot wreck someone's learning path, even if it says something strange on the day.
Measurement	We can count how often the guard overrules the AI. That is a real number in our results.
Demo reliability	Because routing is predictable code, the learning path is the same every run.

Where it appears: PART 4b - when a `[guard_override]` line shows up. It appeared in all three of our live test runs.

Say:

> *"The model proposes and the guard disposes. An LLM suggests the next action, deterministic code validates it against hard rules and overrides it when it is invalid - and we log every override."*

2.4 The safety invariant (our failure never costs the student)

What it is. Two kinds of bad thing can happen when a student submits code:

Their problem	Our problem
Their code has a syntax error	Our sandbox crashed
Their code crashes at runtime	The AI provider timed out
Their answer is wrong	Our database failed
Their code loops forever	The AI returned nonsense

The left column says something about the student, so it changes their mastery. The right column says something about **us**, so it must change nothing.

The clever part: this is not an `if` statement somebody has to remember to write. The two kinds are different *types* in the code, and the function that updates mastery only accepts the first kind. If an infrastructure failure ever reached it, the program would refuse outright.

Where it appears: PART 5, on `[recover] ... mastery_untouched=True`.

Say:

> *"Student outcomes and system faults are disjoint types, and mastery is only reachable from the first. If our infrastructure breaks, the student does not pay for it - and that is enforced by the type system, not by a conditional someone has to notice in review."*

2.5 RAG - teaching from real material

What it is. RAG stands for Retrieval-Augmented Generation. In plain terms: before the AI writes an exercise, we **search our own course notes** for the relevant section and hand it to the AI as source material.

Why it matters. Without it, the AI invents an exercise from whatever it happens to remember. With it, the exercise is grounded in the actual curriculum, and we can show which page it came from.

On screen you will see something like `05_recursion.md#5.1 The idea` - the exact section the exercise was built from.

One detail worth knowing: we do not call RAG at every step. It runs at one point out of twelve, where it helps. If asked why, the answer is that retrieval costs time and adds nothing to arithmetic.

Where it appears: PART 3, and again in PART 4 where retrieval follows the redirect into the functions chapter.

Say:

"Retrieval is filtered by skill first, then searched semantically, and every generated problem records which section it was grounded in."

3. Reading the screen

During the run, lines scroll past. Each is a real decision the program made - not a message we printed in advance.

Line you will see	What it means in plain English
[diagnose]	Working out which skill is weakest and what it depends on
[plan_action]	Choosing topic, difficulty and teaching style for the next task
[retrieve]	Searching our course notes for the right section
[generate_problem]	The AI writing the exercise, grounded in what was retrieved
[execute]	Running the student's code in a sandbox
[misconception]	Working out WHY the answer was wrong - the important one
[mastery]	Updating our estimate of what the student knows
[guard_override]	The guard rejecting the AI's suggestion - point at this
[adapt]	Deciding what to do next
[prereq_redirect]	THE MOMENT. Changing its own goal to a prerequisite
[recover]	An infrastructure failure being absorbed safely
[prereq_return]	Coming back to the original topic
[finalize]	Session finished

The three lines that matter most

- **[misconception]** - says *why* they failed, not just that they failed.
- **[prereq_redirect]** - the agent changing its own objective. The project.
- **[recover] ... mastery_untouched=True** - proof the safety invariant is real.

4. The map - which PART to read when

This is the table you rehearse from. The words are in **CogniFlow_Video_Script.pdf**.

PART	Read it when	You are doing	Length
0	Before recording anything	Setup and three test runs. Nothing spoken.	-
1	Opening shot, seeded model on screen	Pointing at functions 0.55 and recursion 0.35	30s
2	Straight after PART 1, before running anything	Just speaking. No screen action.	20s
3	Immediately after starting 'python run.py live'	Letting the first block scroll; pointing at [diagnose], [retrieve], [mastery]	45s
4	The instant [prereq_redirect] appears	STOP SCROLLING. Hold the screen still.	50s
4b	Only if a [guard_override] line appeared	Pointing at proposed= and final=	10s
5	When [execute] shows sandbox_error	Pointing at the mastery number and holding there	30s
6	When [prereq_return] appears	Scrolling to the learning path summary	25s
7	When the seven [PASS] lines are visible	Showing all seven at once	25s
8	After running 'python run.py ablation'	Showing the results table	35s
9	Final shot	Nothing. Just close.	15s
10	Only if comfortably under 5 minutes	Opening the web UI, starting a session	20s

The single most important instruction in either document

At PART 4, when `[prereq_redirect]` appears, **stop scrolling and leave it on screen** for the whole PART. That line is the entire project. The most common way to ruin this video is to scroll past the best moment in it.

Total timing

PARTs 1 to 9 add up to about **4 minutes 15** of speech. The rulebook allows 3 to 5 minutes, so you have roughly 45 seconds of slack for pauses and for the program thinking. That is comfortable - but not enough to also include PART 10 unless you are running fast. **Going over five minutes is worse than leaving PART 10 out.**

5. Narrating alone

You are doing two jobs at once. Choose your approach during rehearsal, not on the day.

Choose one of these two approaches

Approach	How it works	Trade-off
A - Narrate live	Start the command and talk while output scrolls. Mention the pauses: 'that is a live API call'.	Most impressive, but you must keep up with the output and there are real waits while models respond.
B - Run first, then narrate	Let the whole run finish. Then scroll back through the completed output and narrate over it at your own pace.	Much easier alone, and you control the pace completely. The output is still a real run - nothing is fabricated.

Approach B is not cheating

The rulebook forbids fabricating or manipulating **results**. Scrolling back through the genuine output of a genuine run and explaining it is not that - it is how every code walkthrough in the world is done. What would be wrong is inventing output, or claiming something the screen does not show. **If you are recording alone and nervous, take approach B.** A calm, clear video beats a rushed, authentic-but-flustered one.

Recording in segments

You do not have to do this in one take. Record each PART separately, stop, breathe, then start the next. Join them afterwards in any free editor. This is the single biggest stress reducer available to you, and no one can tell from the finished video.

Six delivery habits

- **Point with the cursor** at the line you are describing. It tells the viewer where to look and keeps you anchored to what is really on screen.
- **Pause after the important sentences.** Silence feels much longer to you than to a viewer. Two seconds after 'It does not move' in PART 5 is right.
- **Slow down at PART 1 and PART 4.** Those two carry the whole idea.
- **Do not read in a monotone.** The script is written the way people speak.
- **If you stumble, stop and redo that PART.** A clean retake costs 30 seconds.
- **Never say a thing the screen is not showing.** If in doubt, describe exactly what is visible. That is always safe and always true.

5b. Actually recording it

Software, settings and the mechanics of joining clips. None of this is difficult, but all of it is easier decided now than at the moment you want to press record.

What to record with

Tool	How	Notes
Windows Game Bar	Press Windows key + G, then the record button. Already installed on Windows 10 and 11.	Easiest option. Records the active window plus your microphone. Saves to Videos/Captures.
OBS Studio	Free from obsproject.com. Add a Display Capture source and an Audio Input source, then Start Recording.	More setup, better control. Worth it only if Game Bar refuses to record your terminal.
Phone on a stand	Point it at the screen.	Last resort. Text will be hard to read - avoid if you can.

Settings that matter

- **Record the whole screen, not a window.** You will switch between the terminal and, for PART 10, a browser.
- **Make the terminal text big.** In PowerShell: Ctrl and the mouse wheel, or right-click the title bar, Properties, Font, size 20 or more. A judge may watch this on a phone.
- **Check the microphone is actually being captured** before the real take. Record ten seconds, play it back, listen. Silent footage is the single most common recording disaster.
- **Turn off notifications.** Windows key + A, then Focus assist / Do not disturb. A message popping up mid-take means recording that PART again.
- **Close everything else** - browser tabs, chat, music. Both for notifications and because a cluttered screen reads as a cluttered project.

Recording PART by PART, and joining the clips

Recording each PART as its own clip is strongly recommended when you are alone. A mistake then costs thirty seconds instead of five minutes.

To join clips	How
Windows Clipchamp	Already installed on Windows 11. Drag clips onto the timeline in order, then Export at 1080p.
Windows Photos	On Windows 10: Photos, New video project, add the clips, Finish video.
Anything you already know	Any editor is fine. No effects, no music, no transitions are needed - plain cuts are correct for a technical demo.

Check the finished length before you submit

The rulebook allows **3 to 5 minutes**. PARTs 1 to 9 come to roughly 4:15 of speech, so a joined video should land near 4:30. If you are over 5:00, drop PART 10 first, then trim silences. **Being over the limit is a rule breach; being at 4:00 is not.**

Before you call it finished

- Watch the whole thing back, once, with sound.
- Can you read the terminal text at normal playback size?
- Is your voice clear and roughly even between clips?
- Is the total between 3:00 and 5:00?
- Does the `[prereq_redirect]` moment stay on screen long enough to read? That is the one shot that must land.
- Are there notification pop-ups or a stray personal window anywhere in frame?

6. Setup - recording day

This is PART 0 of the script, repeated so it is in both documents.

Step 1 - Prove it works before recording anything

In a terminal standing inside your CogniFlow folder - see section 0 if you have not set it up yet:

```
python run.py check      # one real call per AI provider
python run.py live       # the full demo against real models
```

Step 2 - Run it three times, and know what may vary

Run `python run.py live` three times. **Two things must be identical** every time - they are the two the demo actually claims:

Must be identical	What to look for
The learning path	Learning path : recursion -> functions -> recursion
The self-checks	all 7 [PASS] lines, and no [FAIL]

Plenty of other things will differ, and none of them are faults. Measured over three consecutive runs:

- **The exercise titles change.** The AI writes a new exercise each run - that is the point of grounding generation in a model rather than a question bank.
- **The number of steps can change.** The scripted student submits fixed code, and a differently worded exercise may not accept it. In one run a functions answer was graded wrong, and the agent responded with EXPLAIN_DIFFERENTLY and a lower difficulty - an extra loop, and arguably a better demo than the shorter runs.
- **The provider changes.** Groq until its per-minute ceiling, then Google.
- **The duration changes.** 42 to 65 seconds observed.

That variation is the *student* being scripted while the *tutor* is not, which is precisely the claim. **The only thing that should never vary is the learning path itself.** If that changes, or any check reports FAIL, run it twice more before concluding anything - a single odd run is far more likely to be a rate limit than a broken system, and section 0.9 lists what each symptom means.

Step 3 - Screen and audio

- Terminal about 110 characters wide, font large enough to read on a phone.
- Close everything else: notifications, browser tabs, chat apps.
- Dark terminal theme reads better on video than light.
- Record ten seconds and play it back before a real take.

- Keep the script PDF on a second screen or printed - not on the screen you record.

7. If something breaks while recording

The rule

Say what happened, plainly, and keep going. Do not pretend it did not happen and do not talk over it. A recovered failure demonstrates the recovery path we are claiming - genuinely better evidence than a run where nothing went wrong. And if a take is spoiled, just record that PART again.

What you see	What it means	What to say
degraded=True on a line	An AI provider failed, so a built-in template was used	"The provider is rate-limited there, so it fell back to a deterministic template. The tutoring decisions are unaffected - the model does not make those."
provider=google:...	The first provider hit its limit; it failed over automatically	"That is the provider chain failing over live." A feature - mention it.
A long pause	Waiting for a real API response	"That is a live API call - this is not a recording."
An error you do not recognise	Unknown	"Something went wrong there - let me re-run it." Then run 'python run.py verify', which needs no internet.
The learning path came out different	The one thing that should never vary	Stop recording and run it twice more. If it settles, carry on. If it genuinely differs every time, record with 'python run.py verify' instead - see tier 3 in section 0.10 - rather than losing the video.

Known quirk worth understanding

Our first AI provider (Groq) has a free limit of about 8,000 tokens per minute, and one full run uses roughly 35,000. So it may hit the limit partway through and switch automatically to the second provider (Google). You will see `provider=google:gemini-3.5-flash-lite` appear. That is not a fault - it is the failover working, and worth pointing out.

8. Questions a judge may ask

Short honest answers beat long ones. **"I do not know, but I can show you where it is in the code" is a perfectly good answer** and far better than guessing.

Question	Your answer
Is the redirect hardcoded?	No. It is graph traversal over live mastery estimates. There is a test that asserts the state transitions, not the printed text.
Why not just use a big LLM?	An LLM has no calibrated model of the student, so it will happily advance someone who is not ready. The LLM proposes; the guard disposes.
Why not just rules, then?	Rules cannot author a novel exercise grounded in a specific misconception and a specific page of notes. That is what the model is for.
Is the sandbox real?	Yes - a separate process with a timeout and a minimal environment. Student code cannot read our API keys; we verified that with a canary.
Is mastery just a counter?	No, it is Bayesian Knowledge Tracing. It models slipping and guessing and yields a confidence signal the policy consumes.
Did you train a model?	We built parameter fitting for the mastery model. It produced a negative result on held-out data, so we ship the literature defaults and report that.
What happens if the AI is down?	It fails over to a second provider. If all fail, it uses a deterministic template and marks the event degraded - and mastery is untouched, because a provider outage is our fault, not the student's.
Did you write this yourselves?	Yes. Two contributors, both human, and the contributor list is checked automatically before every push.
What does not work yet?	There is no login, no dashboard and no diagnostic quiz - a new student starts from a seeded profile, and nothing is saved between visits. The tutoring engine is complete; the product around it is not. We would rather say that than pretend otherwise.
What breaks it?	Seven and a half percent of students with no gap still get an unnecessary detour. It was twenty-five percent before we required evidence.

9. Glossary

Term	Plain meaning
Agent	A program that decides its own next step instead of following a fixed script.
LangGraph	The library we use to define those steps and the paths between them.
Node	One step in that structure - 'diagnose', 'grade', 'adapt'.
State	Everything the program currently knows about this session.
Checkpoint	A saved copy of that state, written to disk, so it can resume.
Interrupt	The program genuinely stopping to wait for the student - not a loop that spins.
Mastery	Our 0-to-1 estimate of whether the student knows a skill.
Confidence	How much evidence that estimate rests on.
BKT	Bayesian Knowledge Tracing - the method behind those two numbers.
Prerequisite	A skill you must know before another one makes sense.
The guard	Predictable code that checks, and can overrule, the AI's suggestion.
RAG	Searching our own notes and giving them to the AI as source material.
Sandbox	The isolated place student code runs, so it cannot harm anything.
SystemFault	A failure that is ours, not the student's. Never changes mastery.
Ablation	An experiment where you remove one part to prove it was doing something.
Deterministic	Same input, same output, every time. No randomness.
Degraded	A step that fell back to a template because a provider failed.

10. What to study, in priority order

You do not need all of this. If you only do the first three rows, you can still record a good video.

Priority	What	Time	How
Do this first	Section 0 - get the code running on your own machine	30 min	Nothing else in this guide works until it does. Do not leave it to recording day.
Essential	Section 1, until you can say the one idea without reading it	30 min	Read it, then explain it out loud to someone
Essential	Watch a run three times, following the lines in section 3	45 min	'python run.py live' with this guide open beside you
Essential	Read the whole script PDF once, out loud	30 min	Out loud matters - silent reading hides the awkward sentences
Essential	Rehearse PARTs 1 to 9 with the screen, twice	1.5 hours	Time it. Aim for 4:15 of speech.
Strongly	Sections 2.3 and 2.4 - the guard and the safety invariant	45 min	The two a judge is most likely to probe
Strongly	The Q&A table in section 8	30 min	Say the answers aloud; do not just read them
Strongly	Section 5 - decide approach A or B	15 min	Decide in rehearsal, not on the day
If time	Section 2.2 - what BKT actually does	45 min	Optional. The one-sentence version is enough.
If time	Open app/mastery/policy.py and read the rules	1 hour	Ordinary Python with comments. Less scary than it sounds.

A final thought

The strongest thing you can do in a technical Q&A; is be straight about the limits. We know what CogniFlow does not do yet - no login, no dashboard, no diagnostic quiz, nothing saved between visits - and saying so plainly makes every other claim more believable, not less. Confidence is knowing where the edges are, not pretending there are none.

Good luck. The engine works. All you have to do is show it honestly.